Region Affinities

Technical Appendix

Mathematical Framework, Algorithms, and Implementation

Jure Skarabot · May 2026

Companion documents: Summary · Methods Primer · Data Appendix · Region Reference — Identity · Region Reference — Terroir

---

This appendix presents the mathematical framework underlying the Region Affinities tool: the encoding maps for the two layers, the dimensionality reduction and clustering pipelines, the similarity construction used for visualisation, and the formal statistical tests that establish the independence of the two classifications. Notation follows standard linear algebra; treatments are stated rather than derived, with references provided for full derivations. The companion *Methods Primer* provides a non-technical guide to the same material.

A. Mathematical Framework

A.1 Region Universe and Two Encoding Layers

Let $\mathcal{R} = \{r_1, r_2, \ldots, r_n\}$ denote the set of $n = 59$ wine regions. Each region is represented by two independent encodings:

- the *identity vector* $\mathbf{d}_i \in \{-2, -1, 0, +1, +2\}^6$, whose components are integer scores on six anthropological dimensions $\mathrm{D}_1, \ldots, \mathrm{D}_6$;
- the *terroir vector* $\mathbf{t}_i \in \mathbb{R}^{|V|}$, the TF-IDF representation of region i's structured terroir profile in a vocabulary V of bigram features.

The two encodings are computed independently from disjoint source material (cultural narrative versus structured terroir profile) and share no features. The identity matrix $D \in \mathbb{R}^{59\times 6}$ collects all identity vectors row-wise; the terroir matrix $T \in \mathbb{R}^{59\times|V|}$ collects all terroir vectors row-wise.

A.2 TF-IDF Encoding for the Terroir Layer

Let f_{ij} denote the count of term j in the terroir profile of region i. The sublinear term-frequency component is

$$\mathrm{tf}_{ij} = \begin{cases} 1 + \log f_{ij} & f_{ij} > 0, \\ 0 & f_{ij} = 0. \end{cases}$$

The inverse-document-frequency component is

$$\mathrm{idf}_j = \log\frac{1+n}{1+n_j} + 1,$$

where n_j is the number of regions whose profile contains term j. The TF-IDF entry is $T_{ij} = \mathrm{tf}_{ij} \cdot \mathrm{idf}_j$, and rows are subsequently ℓ_2-normalised. Terms span unigrams and bigrams (`ngram_range` $= (1, 2)$), with the vocabulary capped at $|V| = 800$ features by retaining the top-frequency terms across the corpus.

A.3 Standardisation and Dimensionality Reduction

Both layers are standardised before clustering. For each feature column j,

$$\tilde{X}_{ij} = \frac{X_{ij} - \mu_j}{\sigma_j},$$

where μ_j and σ_j are the column mean and standard deviation. Standardisation prevents features with larger numeric scale from dominating distance computations.

Principal Component Analysis (PCA) is then applied. PCA computes the singular value decomposition of the centred data matrix $\tilde{X} = U\Sigma V^\top$, where the columns of V are the principal directions and the columns of $U\Sigma$ are the principal scores. The k-component projection retains the first k columns of the score matrix, capturing the maximum total variance achievable in k dimensions.

For the identity layer, all six principal components are retained, giving a faithful projection that preserves 100% of the variance (the original space is already six-dimensional). For the terroir layer, the first 10 principal components are retained, capturing 32.2% of the total variance — a deliberate compression of the 800-dimensional sparse TF-IDF space into a dense low-dimensional representation suitable for K-means.

A.4 K-Means Clustering

K-means partitions n points $\mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^k$ into K clusters $C_1, \ldots, C_K$ by minimising the within-cluster sum of squares:

$$\min_{C_1,\ldots,C_K} \sum_{c=1}^{K} \sum_{\mathbf{x}_i \in C_c} \|\mathbf{x}_i - \boldsymbol{\mu}_c\|_2^2, \qquad \boldsymbol{\mu}_c = \frac{1}{|C_c|} \sum_{\mathbf{x}_i \in C_c} \mathbf{x}_i.$$

The standard Lloyd algorithm alternates two steps until convergence: assign each point to the nearest centroid, then recompute centroids as cluster means. The objective is non-convex, and the result depends on initialisation; the implementation uses K-means++ seeding with `n_init` $= 20$ restarts and returns the partition with the smallest objective. The random seed is fixed at `random_state` $= 42$ for reproducibility.

The number of clusters K is chosen separately for each layer: $K_{\text{id}} = 6$ for the identity layer and $K_{\text{terr}} = 7$ for the terroir layer, both selected by silhouette-score maximisation across $K \in \{3, \ldots, 10\}$.

A.5 Silhouette Score for Cluster Quality

For each point $\mathbf{x}_i$ assigned to cluster C_c, define

$$a(i) = \frac{1}{|C_c| - 1} \sum_{\substack{\mathbf{x}_j \in C_c \\ j \neq i}} d(\mathbf{x}_i, \mathbf{x}_j), \qquad b(i) = \min_{c' \neq c} \frac{1}{|C_{c'}|} \sum_{\mathbf{x}_j \in C_{c'}} d(\mathbf{x}_i, \mathbf{x}_j),$$

where d is Euclidean distance. The silhouette of point i is

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, +1].$$

Values near $+1$ indicate well-clustered points; values near 0 indicate points on cluster boundaries; values near -1 indicate misclassification. The overall silhouette score is the mean of $s(i)$ over all n points.

A.6 Adjusted Rand Index for Clustering Agreement

Let $U = \{U_1, \ldots, U_K\}$ and $V = \{V_1, \ldots, V_L\}$ be two partitions of the same n objects. Let $n_{kl} = |U_k \cap V_l|$ be the size of the intersection, $a_k = |U_k|$, and $b_l = |V_l|$. The Rand Index counts the fraction of object pairs on which the two partitions agree (both same-cluster or both different-cluster). The Adjusted Rand Index (ARI) corrects for chance agreement under a null model of randomly shuffled labels:

$$\mathrm{ARI}(U, V) = \frac{\sum_{k,l} \binom{n_{kl}}{2} - \frac{1}{\binom{n}{2}} \sum_k \binom{a_k}{2} \sum_l \binom{b_l}{2}}{\frac{1}{2}\left[\sum_k \binom{a_k}{2} + \sum_l \binom{b_l}{2}\right] - \frac{1}{\binom{n}{2}} \sum_k \binom{a_k}{2} \sum_l \binom{b_l}{2}}.$$

ARI takes values in $[-1, +1]$: $+1$ for identical partitions, 0 for the expected value under random labelling, and negative values for partitions that disagree more than chance. ARI is the standard measure for comparing clusterings of differing K and L on the same objects.

A.7 Chi-Squared Test of Independence

The $K \times L$ contingency table N with entries n_{kl} records the joint cluster membership of the n objects under the two partitions. Under the null hypothesis that the two cluster labels are independent, the expected count for cell (k, l) is

$$E_{kl} = \frac{a_k \, b_l}{n}.$$

Pearson's chi-squared statistic is

$$\chi^2 = \sum_{k=1}^{K} \sum_{l=1}^{L} \frac{(n_{kl} - E_{kl})^2}{E_{kl}},$$

which is asymptotically distributed as $\chi^2_{(K-1)(L-1)}$ under the null. For $K = 6$, $L = 7$, the degrees of freedom are $(6 - 1)(7 - 1) = 30$.

A.8 Cosine Similarity and the kNN Graph

For visualisation, both layers are converted to similarity networks. Pairwise cosine similarity between region vectors $\mathbf{x}_i, \mathbf{x}_j$ is

$$\cos(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{x}_i^\top \mathbf{x}_j}{\|\mathbf{x}_i\|_2 \|\mathbf{x}_j\|_2}.$$

For the identity layer, $\mathbf{x}_i = \mathbf{d}_i$ (the raw D-score vector). For the terroir layer, $\mathbf{x}_i = \mathbf{t}_i$ (the ℓ_2-normalised TF-IDF vector). Each region is then connected to its $k = 5$ most similar peers; the symmetrised union of these neighbourhoods defines the edge set of the corresponding similarity graph. The two graphs are presented in the Dual Networks view of the tool.

B. Algorithms

Algorithm 1 Identity-layer pipeline

Require: D-score matrix $D \in \mathbb{R}^{59\times 6}$
$\tilde{D} \leftarrow \textsc{StandardScaler}(D)$
$(U, \Sigma, V^\top) \leftarrow \mathrm{SVD}(\tilde{D})$
$Z \leftarrow U_{:,1:6}\,\Sigma_{1:6,1:6}$ ▷ 6 PCs, 100% variance
$\mathbf{c}_{\mathrm{id}} \leftarrow \textsc{KMeans}(Z,\ K = 6,\ \texttt{n\_init}{=}20,\ \mathrm{seed}{=}42)$
$S_{\mathrm{id}} \leftarrow$ pairwise cosine similarity on D
$G_{\mathrm{id}} \leftarrow$ symmetrised k-NN graph from S_{id} with $k = 5$
return $(\mathbf{c}_{\mathrm{id}}, G_{\mathrm{id}})$

Algorithm 2 Terroir-layer pipeline

Require: Terroir profile texts $\{p_i\}_{i=1}^{59}$
$T \leftarrow \textsc{TfidfVectoriser}(\{p_i\},\ \mathrm{ngram} = (1,2),\ \max\_features = 800,\ \mathrm{sublinear\_tf{=}True})$
$\tilde{T} \leftarrow \textsc{StandardScaler}(T)$
$(U, \Sigma, V^\top) \leftarrow \mathrm{SVD}(\tilde{T})$
$Z \leftarrow U_{:,1:10}\,\Sigma_{1:10,1:10}$ ▷ 10 PCs, 32.2% variance
$\mathbf{c}_{\mathrm{terr}} \leftarrow \textsc{KMeans}(Z,\ K = 7,\ \texttt{n\_init}{=}20,\ \mathrm{seed}{=}42)$
$S_{\mathrm{terr}} \leftarrow$ pairwise cosine similarity on T
$G_{\mathrm{terr}} \leftarrow$ symmetrised k-NN graph from S_{terr} with $k = 5$
return $(\mathbf{c}_{\mathrm{terr}}, G_{\mathrm{terr}})$

Algorithm 3 Independence test

Require: Cluster labels $\mathbf{c}_{\mathrm{id}}, \mathbf{c}_{\mathrm{terr}}$ for $n = 59$ regions
$N \leftarrow$ contingency table of $\mathbf{c}_{\mathrm{id}}$ against $\mathbf{c}_{\mathrm{terr}}$ ▷ 6×7
$E_{kl} \leftarrow a_k b_l / n$ for all k, l
$\chi^2 \leftarrow \sum_{k,l} (N_{kl} - E_{kl})^2 / E_{kl}$
$p \leftarrow P(\chi^2_{30} > \chi^2)$
$\mathrm{ARI} \leftarrow \textsc{AdjustedRandIndex}(\mathbf{c}_{\mathrm{id}}, \mathbf{c}_{\mathrm{terr}})$
return $(\chi^2, p, \mathrm{ARI})$

C. Implementation

C.1 Technology Stack

The analytical pipeline is implemented in Python using `scikit-learn` for TF-IDF vectorisation, PCA, K-means, and silhouette computation; `scipy.stats` for the chi-squared test; and `numpy` for linear algebra. The Region Affinities web tool is a single-page React application built with `vite`, with interactive force-directed graph layouts produced by `d3.js` (v7) using the Fruchterman–Reingold algorithm. All cluster assignments, edge lists, and similarity scores are pre-computed and shipped as static JSON; no server-side computation occurs at runtime.

C.2 Computational Pipeline

1. *Encoding.* D-scores are loaded from a curated JSON file. Terroir profiles are loaded as plain text and tokenised by `TfidfVectoriser`.
2. *Standardisation and PCA.* `StandardScaler` is applied independently to each layer; PCA is computed via `sklearn.decomposition.PCA`.
3. *Clustering.* `KMeans` is invoked twice, once per layer, with `n_init` = 20, `random_state` = 42.
4. *Similarity and graph construction.* Pairwise cosine similarities are computed via `sklearn.metrics.pairwise.` the k-NN graph is built by selecting the top-5 neighbours of each region and symmetrising the resulting edge set.
5. *Statistical tests.* ARI is computed via `sklearn.metrics.adjusted_rand_score`; the chi-squared test via `scipy.stats.chi2_contingency`.
6. *Export.* All cluster labels, edge lists, and similarity matrices are serialised to JSON files for the front-end.

C.3 Force-Directed Layout (Front-End)

The `d3.js` simulation combines four forces: a link force with distance proportional to the inverse cosine similarity (highly similar regions are drawn closer), a repulsive charge force, a centring force, and a collision-avoidance force based on node radius. Node radius is fixed; node colour encodes the cluster assignment of the relevant layer.

C.4 Reproducibility

The analytical pipeline is deterministic given the random seed (`random_state` = 42). All input data, cluster labels, and statistical test results are reproduced exactly on rerun. Versions: Python 3.11, scikit-learn 1.4, scipy 1.12, numpy 1.26.

References

Hubert, L. & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2(1), 193–218.

Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.

Arthur, D. & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. *Proc. SODA 2007*, 1027–1035.

Pearson, K. (1900). On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. *Philosophical Magazine*, 50(302), 157–175.

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.

Salton, G. & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.